

3-3 · DuckDB를 이용한 로컬 대용량 데이터 처리 · 실습 1.5H

대용량 변환 실습

COPY · 크기·속도 실측

CSV 를 Parquet 로 — 크기와 속도를 직접 쟀다

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① speed.py 에 이어 붙이며 COPY 로 zstd Parquet 를 만든다
- ② 크기 배율과 같은 질의의 속도 배율을 3회 평균으로 쟀다
- ③ count(*) 비교가 거꾸로 나오는 이유를 보고 다섯 열로 다시 쟀다
- ④ 압축 3종을 비교하고 PARTITION_BY 로 값별 폴더를 만든다

쟀 수치는 실측기록_3-3.md 에 적어 정리·회고에서 채팅으로 비교합니다

일곱 단계 — 변환하고, 재고, 이유를 확인하고, 기록

speed.py 한 파일에 1~6단계를 이어 붙이고 매번 `python3 speed.py — bench()` 는 3단계에서 정의

1. COPY 변환

speed.py 시작
zstd

2.3. 크기·속도

4.1배 작아짐
질의 30~60배

4. 왜 빠르냐

count(*) 함정
→ 다섯 열

5.6. 압축·파티션

3종 비교
PARTITION_BY

7. 실측 기록

실측기록_3-3.md
편집기로

핵심 명령 — 변환 · 크기 · 속도

하려는 일	코드	확인
Parquet 변환	COPY (SELECT * FROM 'events.csv') TO 'events.parquet' (FORMAT parquet, COMPRESSION zstd)	「변환 완료」
크기 비교	ls -lh events.csv events.parquet · os.path.getsize()	41M → 9.8M 24.2% · 4.1배
속도 재기	def bench(sql, rounds=3) — 3단계에서 정의	4~6단계도 쓴다
같은 질의 둘	SELECT event_type, sum(value) ... GROUP BY event_type	Parquet 30~60배 빠름

1000분의 몇 초를 재니 흔들리는 것이 정상 — 화면 예: CSV 0.228 초 vs Parquet 0.007 초, 30.8배

「전부 읽기」가 더 빠르다? — 측정 방법을 의심

1 한 열 0.0018 초 · **count(*)** 0.0019 초
count(*) FROM (SELECT *) 로 켜 「전부 읽기」가 같거나 오히려 빠르다

2 DuckDB 가 **SELECT *** 를 걷어 냈다
Parquet 꼬리말의 행 수만 읽었다 — 열을 하나도 읽지 않은 것

3 다섯 열을 실제로 쓰게 — 열마다 집계 하나
다섯 열 0.0150 초, 한 열의 8.1배(돌릴 때마다 4~8배)

4 **CSV** 는 한 열 0.208 · 다섯 열 0.257 초
어느 쪽이든 글자 전체를 읽어야 해 차이가 작다

거꾸로 나오면 EXPLAIN 을 붙여 본다 — sum(value) 는 value 한 열만 읽고, count(*) 는 1행 결과만 남는다

5단계 — 압축 3종 비교, 기본값은 snappy

COMPRESSION	크기	확인 포인트
zstd	9.8 MB	가장 작다 — 오늘의 기본
snappy	15.9 MB	안 적으면 이것(DuckDB 기본)
gzip	10.6 MB	zstd 다음으로 작다

속도는 셋 다 0.004~0.005 초 — 이 크기에선 차이가 없다

압축을 적지 않은 COPY 는 snappy(15.9 MB 쪽)로 저장된다 — 오늘처럼 COMPRESSION zstd 를 적는다

6단계 뒤 — 옵션을 빼고 같은 폴더에 다시 쓴다

speed.py 에 붙이면 그 뒤로 계속 멈춥니다 — parted_again.py 단독으로 한 번만

PARTITION_BY	find · du -sh	parted_again.py	실패	해결 둘
speed.py 6단계 OVERWRITE_OR_IGN ORE	폴더 셋 · 10M event_type=buy/...	옵션을 뺀 COPY 단독으로 한 번	Directory "events_parted" is not empty!	OVERWRITE 또는 OVERWRITE_OR_IGN ORE

폴더 이름이 곧 조건 — buy 만 읽을 때 다른 폴더는 안 연다

1 FROM 'events_parted/*/*.parquet' WHERE event_type = 'buy'
파티션됨 0.0031 초 — view.cart 폴더는 아예 열지 않는다

2 FROM 'events.parquet' WHERE event_type = 'buy'
단일 파일 0.0037 초 — 전체에서 골라낸다

3 비슷하거나 파티션 쪽이 조금 빠르다
100만 행은 작아 차이가 작다 — 수억 행에서는 결정적

4주차 예고

Iceberg 가
이 원리를 정교하게

7단계 — 실측기록_3-3.md 를 편집기로 week3 에

항목	CSV	Parquet(zstd)	배율
파일 크기	MB	MB	배
집계 질의	초	초	배
한 열만 읽기	초	초	—
다섯 열 읽기	초	초	—

압축 비교(zstd·snappy·gzip MB) · count(*) 가 더 빨랐던 이유 · 결론 한 문장

수치는 환경마다 다르다 — 배율을 채팅에 올려 비교. 중요한 것은 방향: Parquet 가 작고 빠르다, 열을 적게 읽을수록 유리하다

이 1.5시간이 끝날 때 확인할 것

- 1 events.parquet 9.8M — CSV 41M 의 24.2%(4.1배)
- 2 같은 집계 질의가 Parquet 에서 수십 배 빠르다(30~60배)
- 3 count(*) 가 한 열과 같거나 빠른 이유를 말할 수 있다
- 4 다섯 열은 한 열의 4~8배 · CSV 는 차이가 작다
- 5 zstd 9.8 · snappy 15.9 · gzip 10.6 MB, 기본은 snappy
- 6 not empty! 를 봤고, 실측기록_3-3.md 를 적었다

3-3 · 실습 1.5H 마무리

다음은 정리·회고 (0.5H)

DuckDB 문법을 한 장으로 정리하고, 실측 수치를 채팅에 올려 비교합니다

✓ DuckDB 요약 표 — 하려는 일과 코드

✓ 실측 수치 공유 — 채팅으로

✓ 자주 겪는 오류 네 가지

✓ 3-4 준비 — 파일 넷 확인

가져갈 것

실측기록_3-3.md · events.parquet